

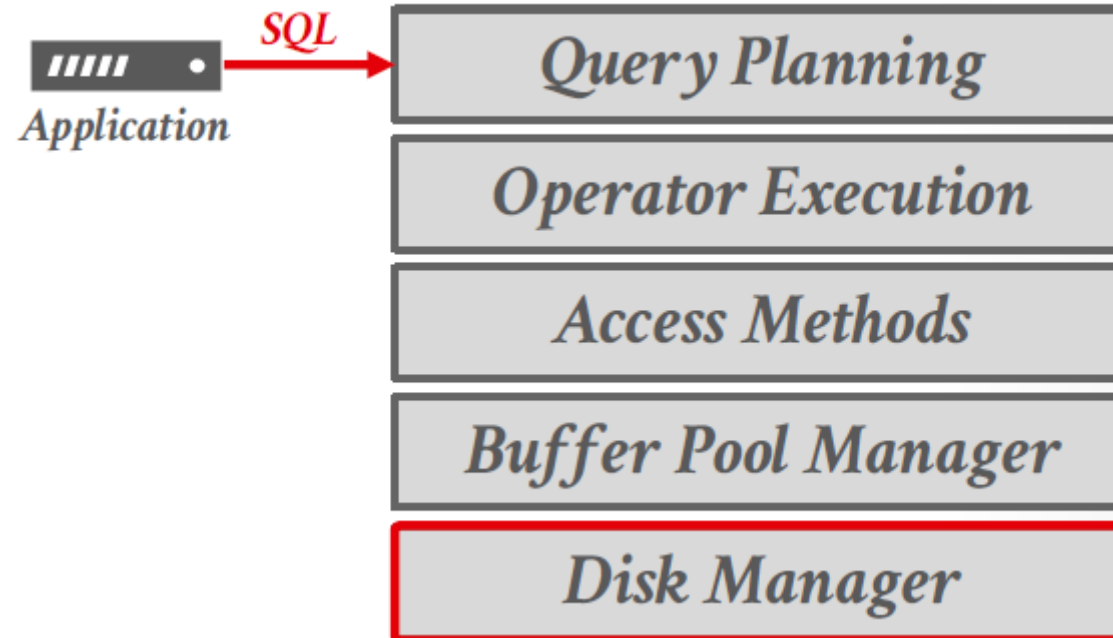
Curso _____
Estructuras de datos

**UNIVERSIDAD
DE ANTIOQUIA**

Clase 4 – Archivos (Parte 2)

Sobre el curso

- En este curso se abordaran algunos conceptos relacionados con el diseño y la implementación de sistemas de gestión de bases de datos orientados a disco.
- No es un curso sobre cómo usar una base de datos para construir aplicaciones ni sobre cómo administrar una base de datos.

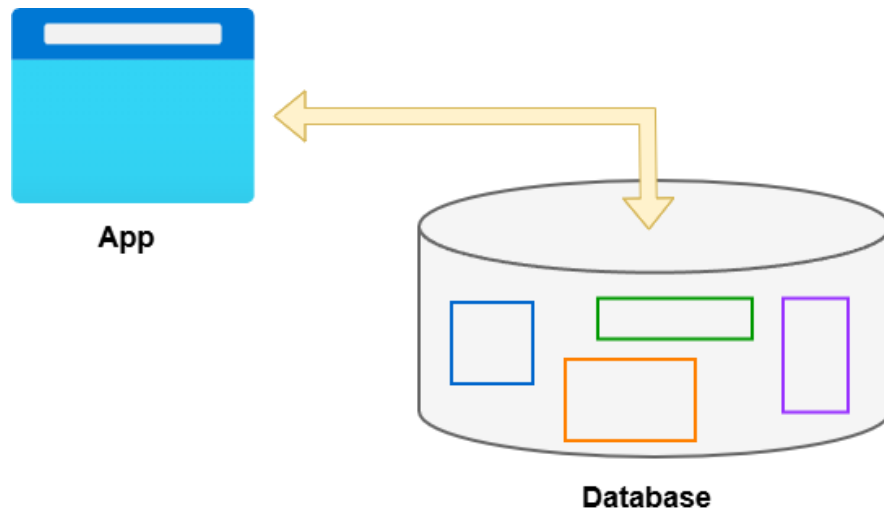


Base de datos

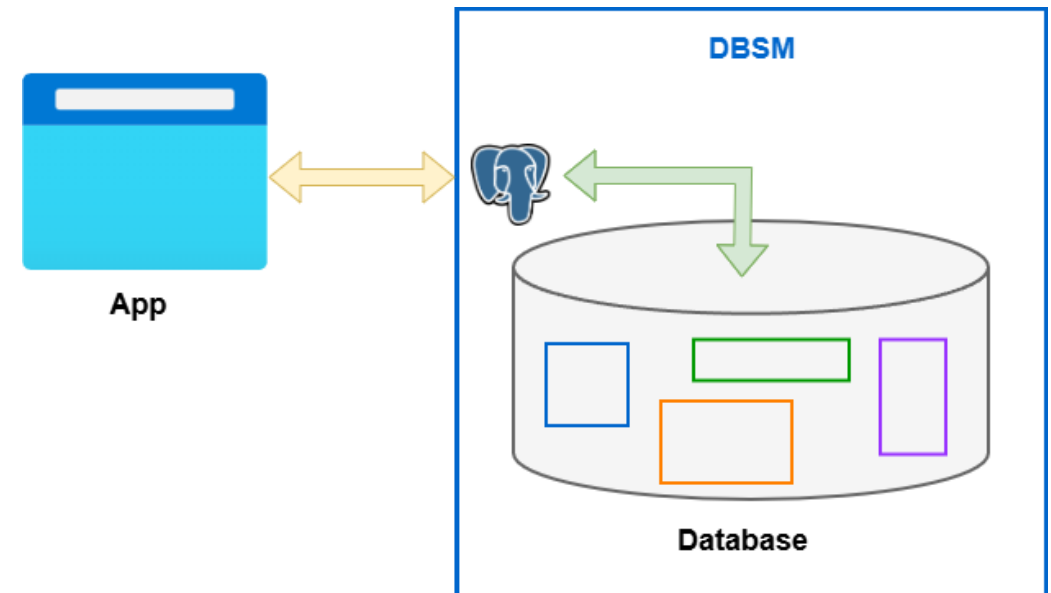
La clase anterior tratamos algunos de los siguientes conceptos

- Base de datos
- Manejo de bases de datos:
 - **Forma antigua:** Sistemas basados en archivos
 - **Actual:** Motor de bases de datos (DBSM)

Sistemas basados en archivos



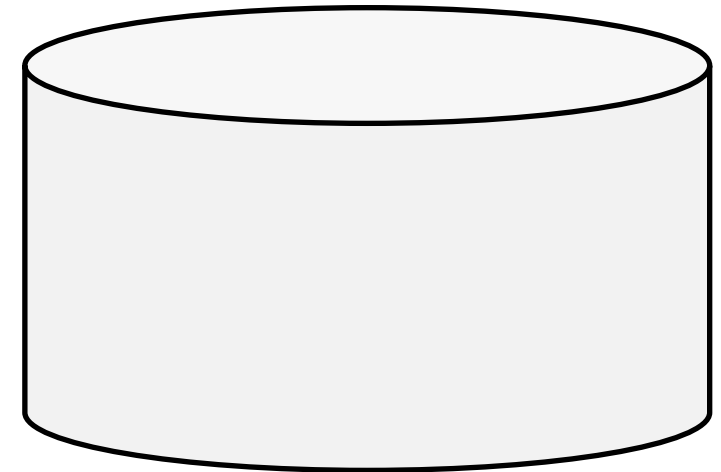
Motor de bases de datos



Base de datos

¿Cómo se representan los datos en un DBMS?

ID	Last Name	First Name	Age	Salary
123	Simpson	Homer	31	\$400
443	Simpson	Marge	32	\$140
244	Flanders	Ned	55	\$300
134	Skinner	Seymour	55	\$400



Representación de los datos

Conceptos claves

Archivo (File DB)

Pagina (Page)

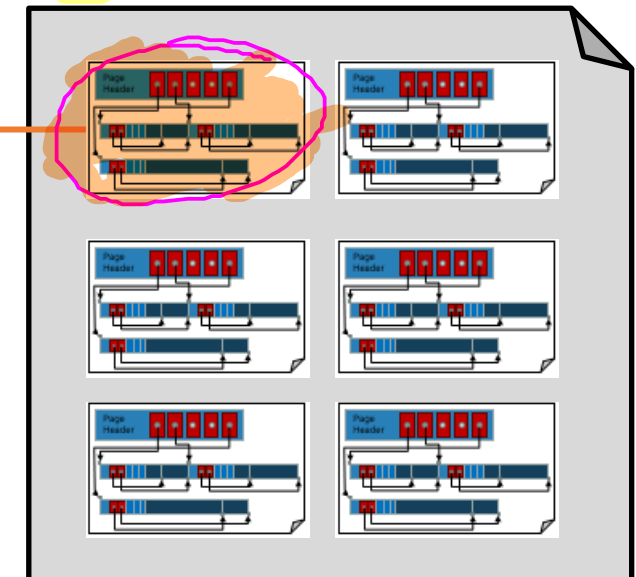
Registro (Record)

Campo (Field)

Relation (table)

ID	Last Name	First Name	Age	Salary
123	Simpson	Homer	31	\$400
443	Simpson	Marge	32	\$140
244	Flanders	Ned	55	\$300
134	Skinner	Seymour	55	\$400

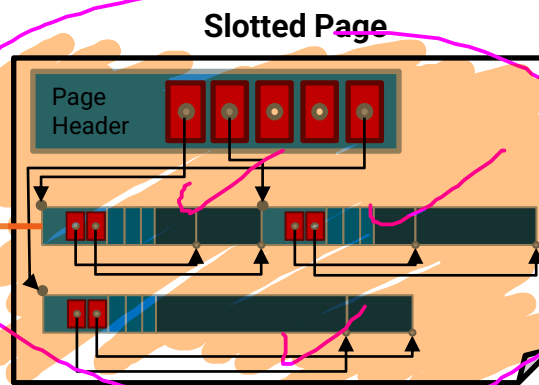
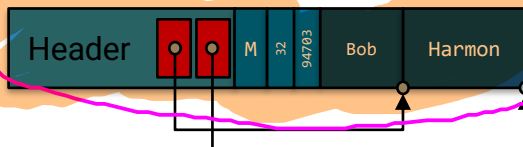
File



Record

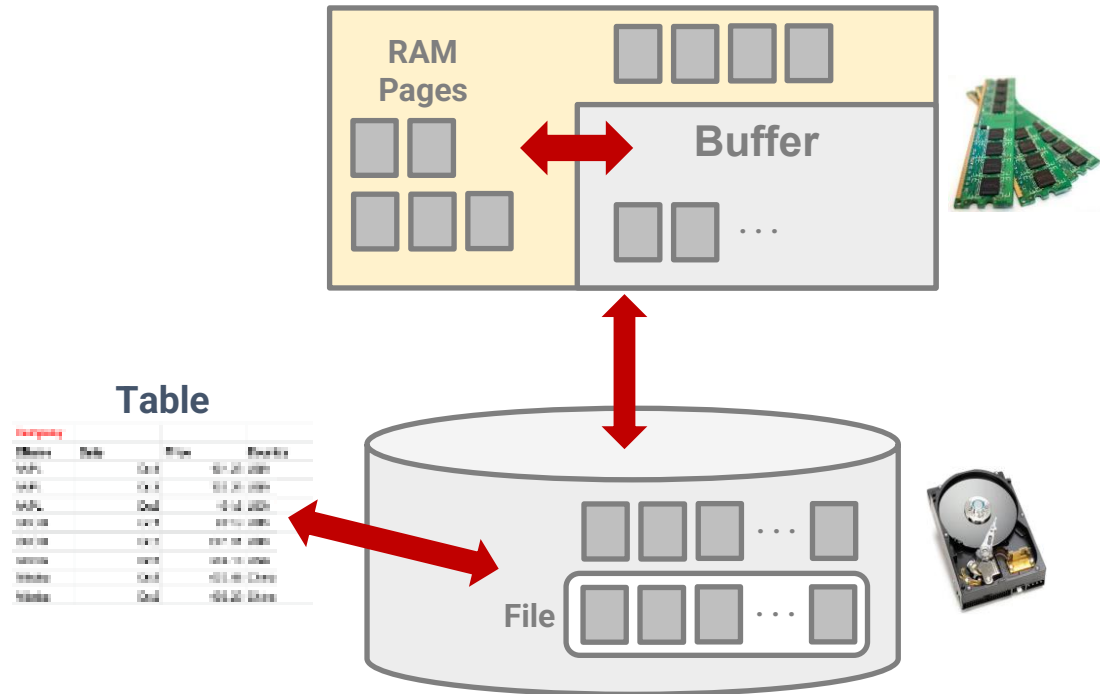
Bob	Harmon	M	32	94703
varchar	varchar	char	int	int

Byte Representation of Record



Buffer Pool

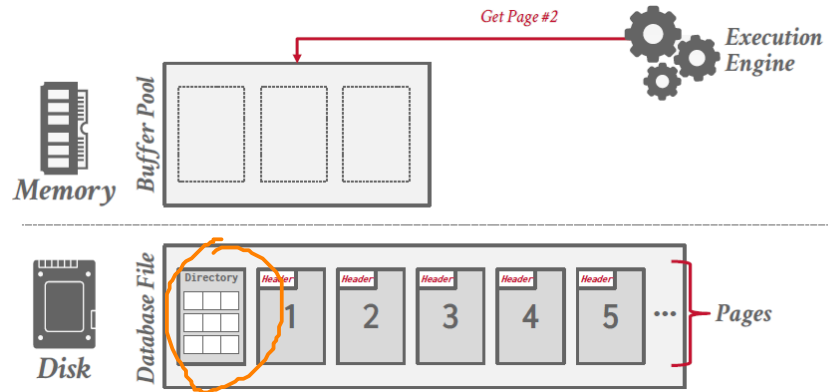
Conceptos claves



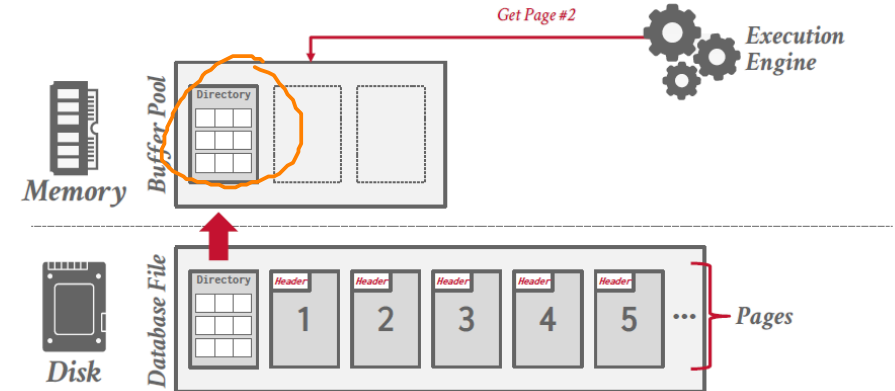
Buffer Pool

Conceptos claves

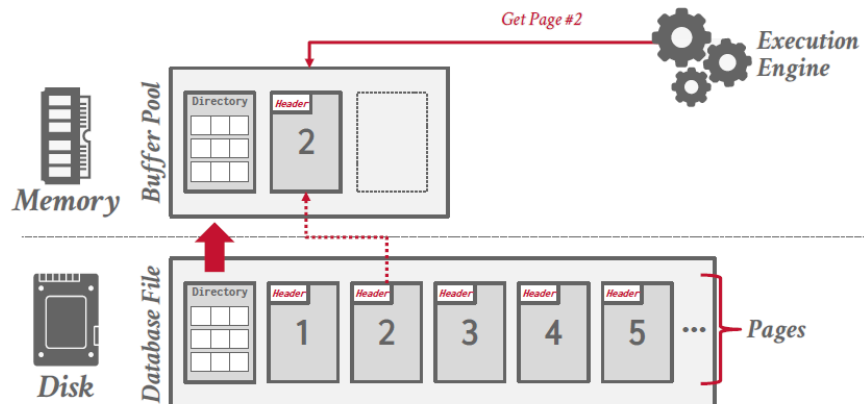
1 Solicitud de Datos por el Motor de Ejecución



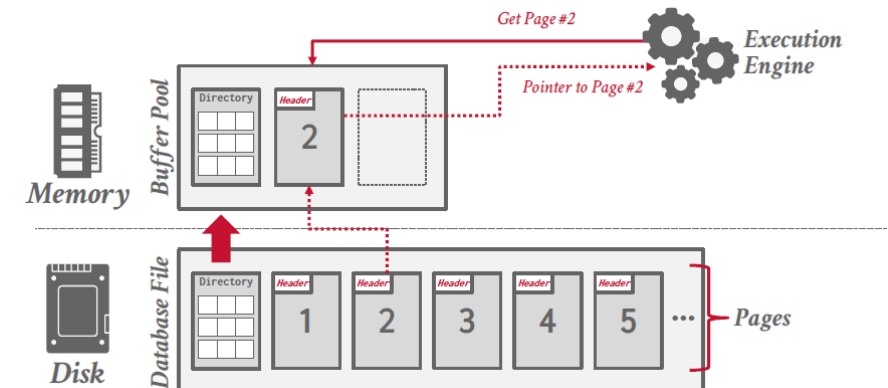
2 Verificación del Directorio de páginas y Detección de Buffer Miss



3 Operación de E/S: Recuperación del Bloque Físico



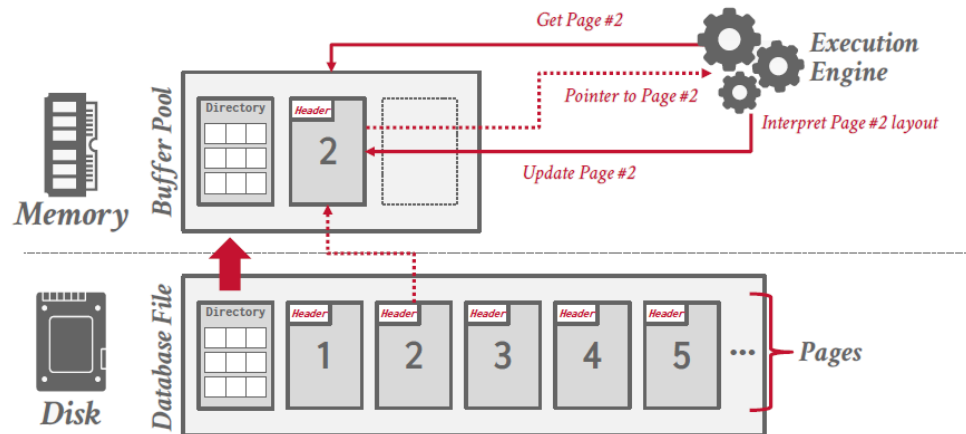
4 Operación de E/S: Recuperación del Bloque Físico



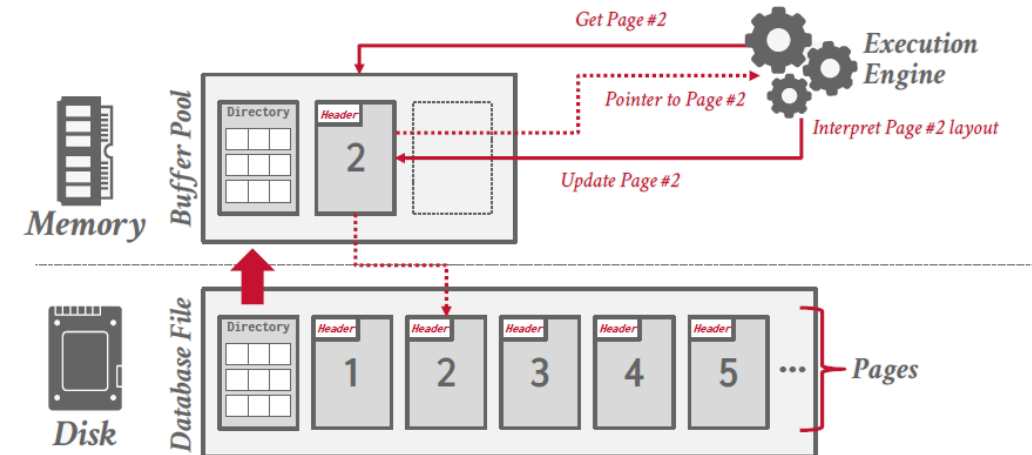
Buffer Pool

Conceptos claves

5 Vinculación Lógica mediante Punteros de Memoria

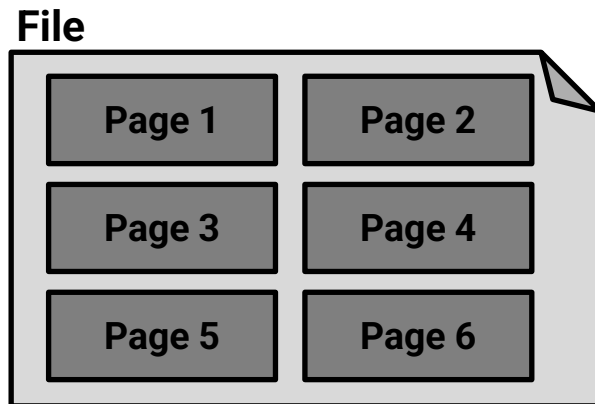


6 Actualización de Registros y sincronización

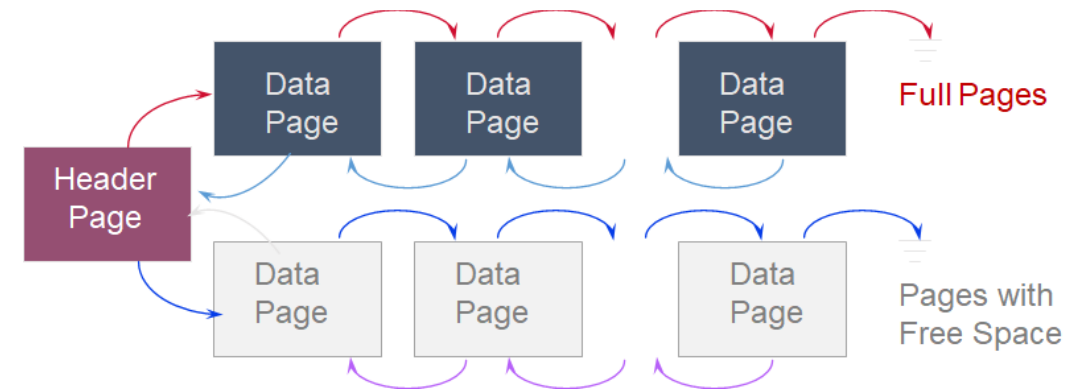


Archivo

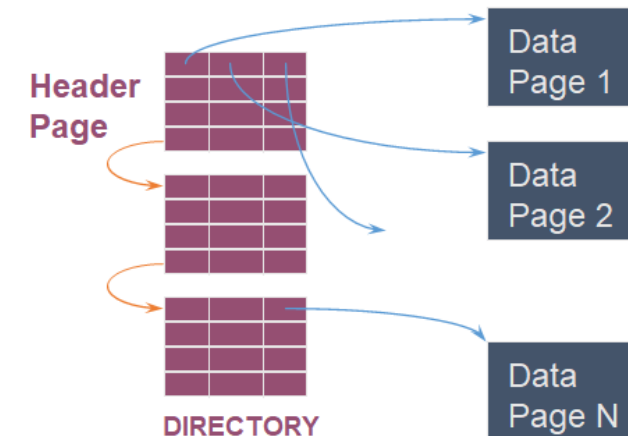
Archivos no ordenados (Unordered heap files)



Lista
enlazada

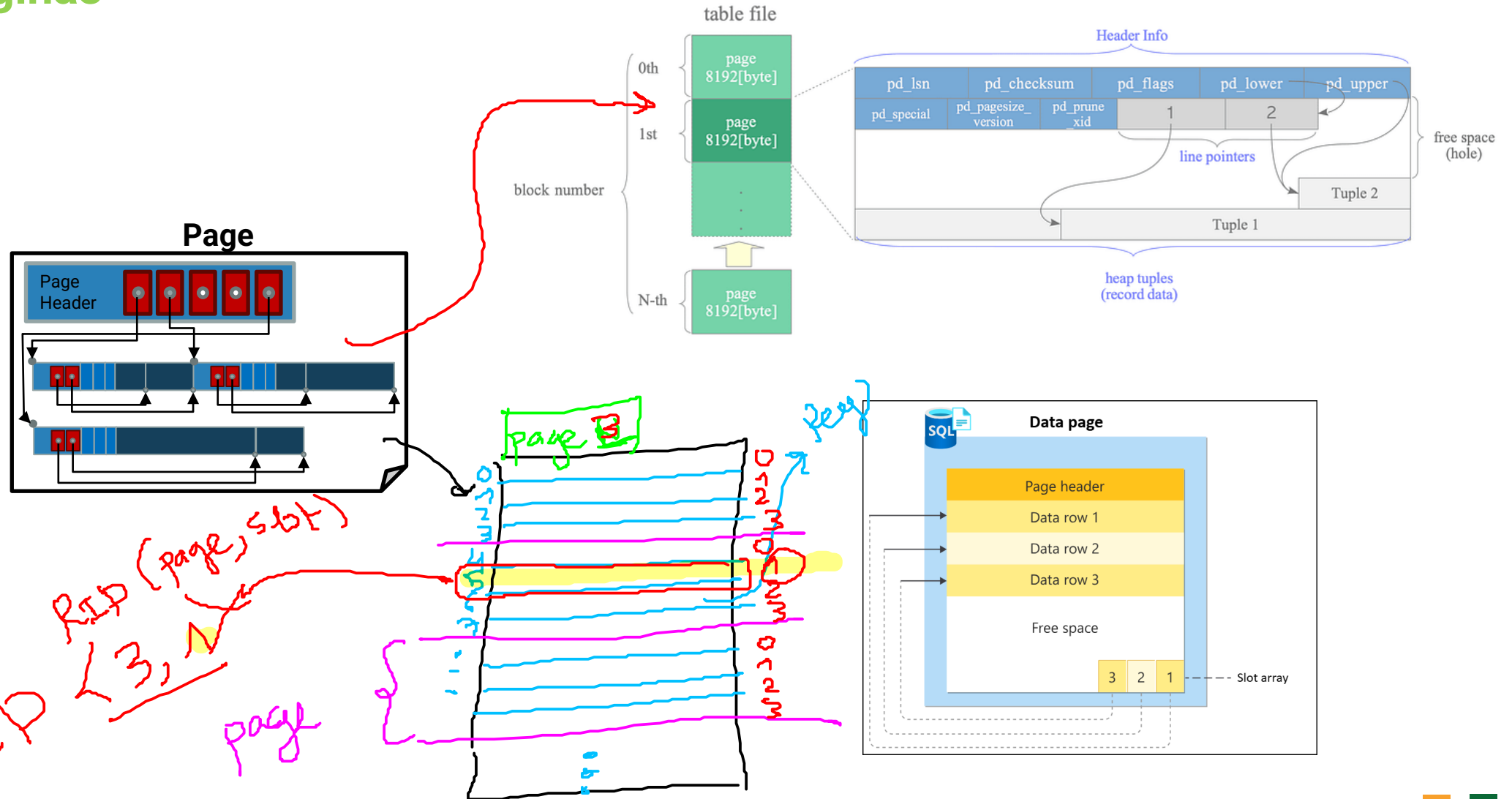


Lista de
directorios



Archivo

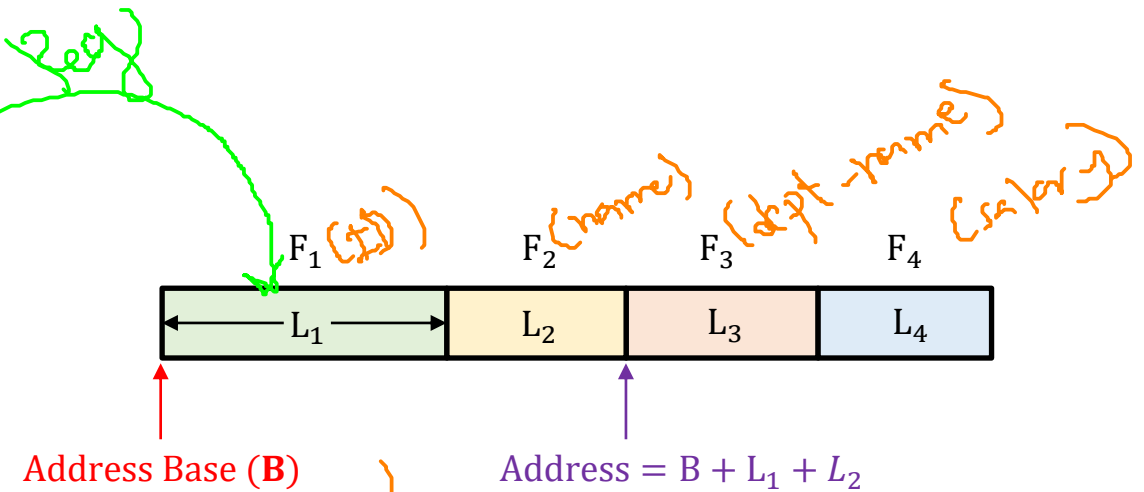
Paginas



Archivo

Registros

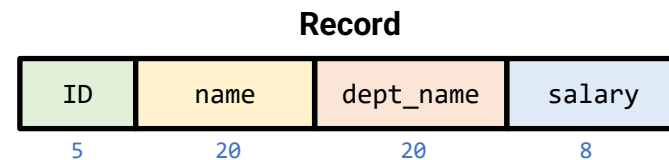
ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000



```

type instructor = record
  ID : varchar (5);
  name : varchar(20);
  dept_name : varchar(20);
  salary : numeric (8,2);
end;

```



= 53 bytes

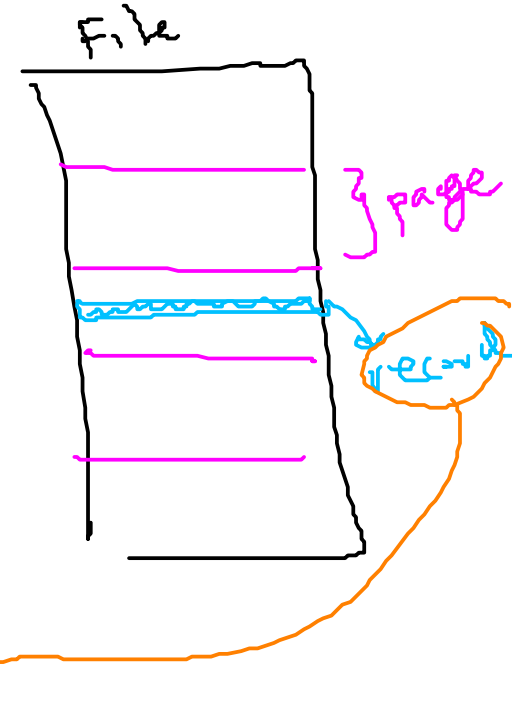
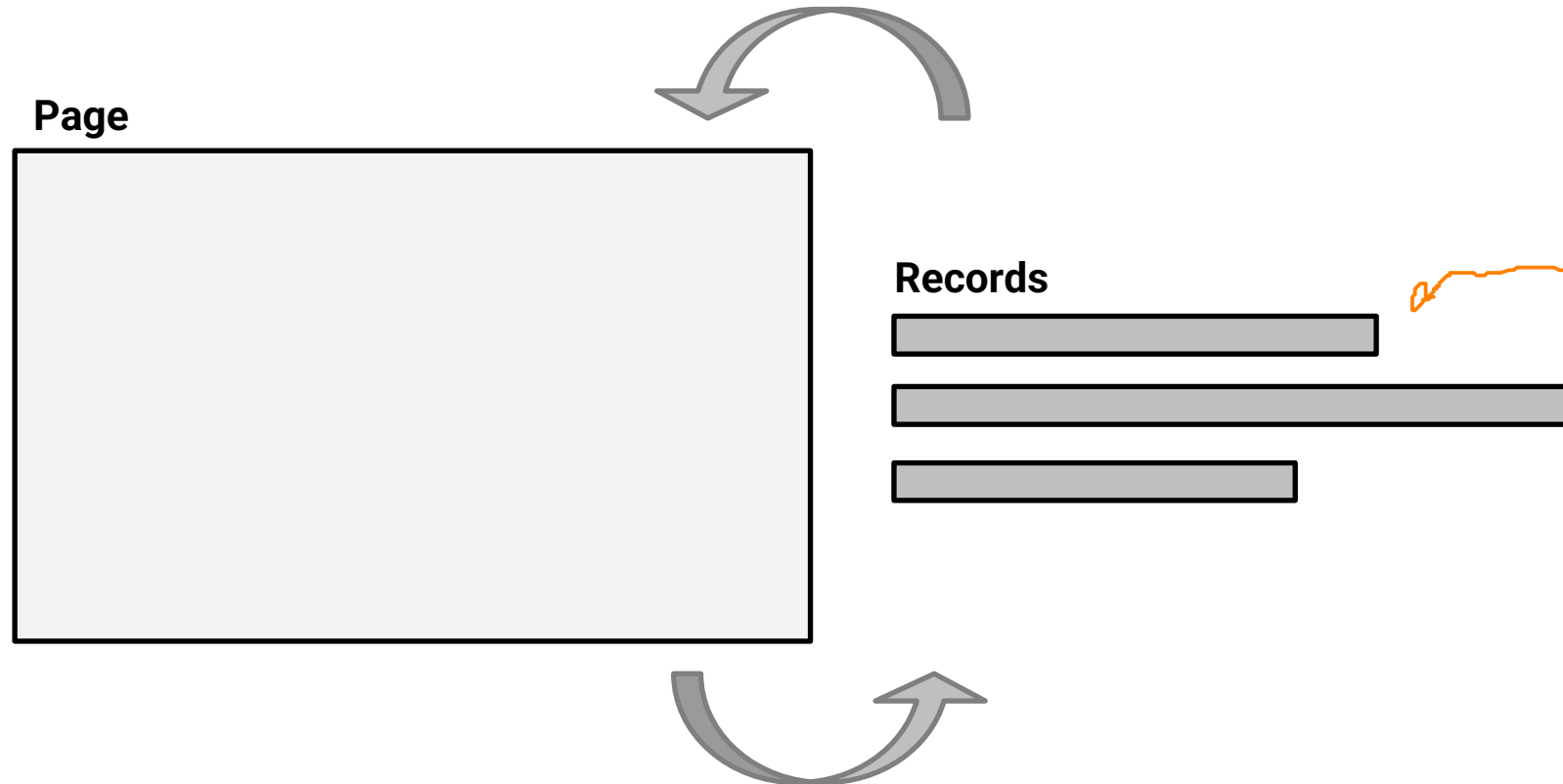


Registros

Definición del problema

Pregunta: ¿Cómo almacenar registros en una página/archivo?

- Se pueda apuntar a cada uno de los registros
- Sea posible insertar/eliminar registros empleando pocos accesos al disco

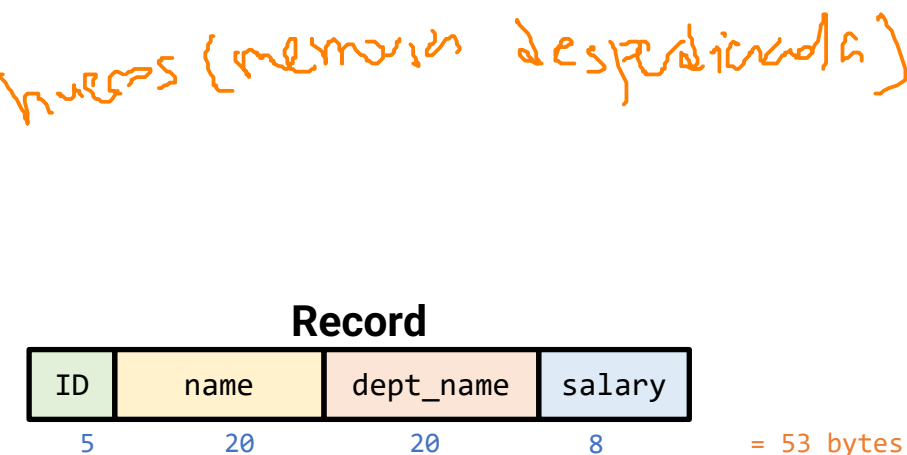


Formato de los registros

- **Modelo Relacional** →
 - Cada registro en una tabla tiene un **tipo de estructura fijo**.
- Se asume que el **Catálogo del Sistema** almacena el **esquema**
 - No es necesario almacenar la información de tipo junto con cada registro (*ahorra espacio*).
 - El catálogo es simplemente **otra tabla del sistema**.
- **Objetivos:**
 - Los registros deben ser **compactos** tanto en memoria como en disco.
 - Permitir **acceso rápido a los campos**
- **Casos**
 - **Sencillo:** Campos de **longitud fija (FLR)**.
 - **Desafío:** Campos de **longitud variable (VLR)**.

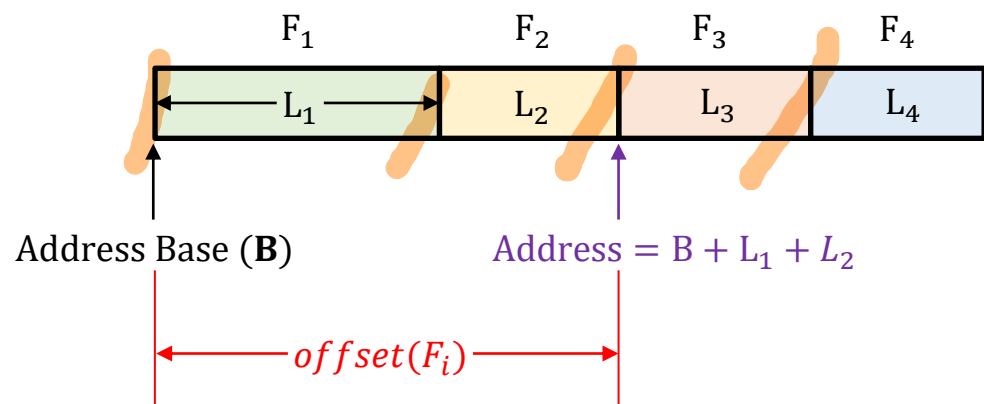
✓ Esquema

```
type instructor = record
  ID : varchar (5);
  name : varchar(20);
  dept_name : varchar(20);
  salary : numeric (8,2);
end;
```



Registros de longitud fija (FLR)

- Los **tipos de campo** son los mismos para todos los registros en un archivo.
 - La **información de tipo** se almacena separadamente en el catálogo del sistema.
- En disco, la representación en bytes es la misma que en memoria.
- ¿Cómo encontrar el campo ***i-ésimo***?
 - Se calcula mediante operaciones aritméticas sobre desplazamientos (**offsets**) (*rápido*).
 - **P**: ¿Es compacto? → **R**: No, fragmentación interna
 - **P**: ¿qué ocurre con los valores nulos? → **R**: Es obligatorio reservar el espacio



$$\text{offset}(F_i) = B + \sum_{j=1}^{i-1} L_j$$

- **B (Address Base)**: Dirección donde comienza el registro completo en la memoria o el disco.
- **L_j (Length)**: Longitud (en bytes) de cada campo previo.



* Continuación: 13/03/2026

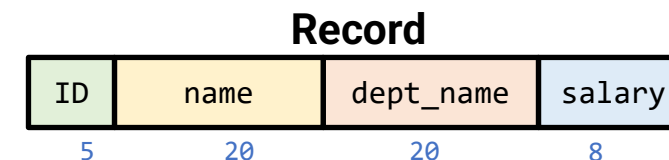
Registros de longitud fija (FLR)

Ejemplo: Retomemos nuevamente la tabla de profesores y el esquema definido en la figura:

- Si el registro del instructor "Einstein" comienza en la dirección de memoria 1000, ¿en qué dirección exacta comienza el campo salary? (**Rta:** 1045)
- Si quiero leer únicamente el dept_name, ¿cuántos bytes debe ignorar (saltar) el sistema desde el inicio del registro? (**Rta:** 25)
- Si el nombre del instructor es "Wu" (solo 2 letras), ¿cuántos bytes ocupa ese registro en el disco? ¿Sigue midiendo 53 bytes o se reduce? (**Rta:** 53)
- Cuántos bytes de "relleno" (basura o espacios) se están desperdiciando en el campo name si guardamos a "Einstein" (8 letras)? (**Rta:** 12)
- Qué sucede si intentamos contratar a un instructor cuyo nombre tiene 25 letras? ¿El registro crece a 58 bytes o el nombre se corta? (**Rta:** Se corta)
- Si el campo salary es opcional (puede ser NULL), ¿el tamaño total del registro pasaría a ser de 45 bytes (53 - 8)? (**Rta:** Sigue siendo el mismo).

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

```
type instructor = record
  ID : char(5);
  name : char(20);
  dept_name : char(20);
  salary : numeric(8,2);
end;
```



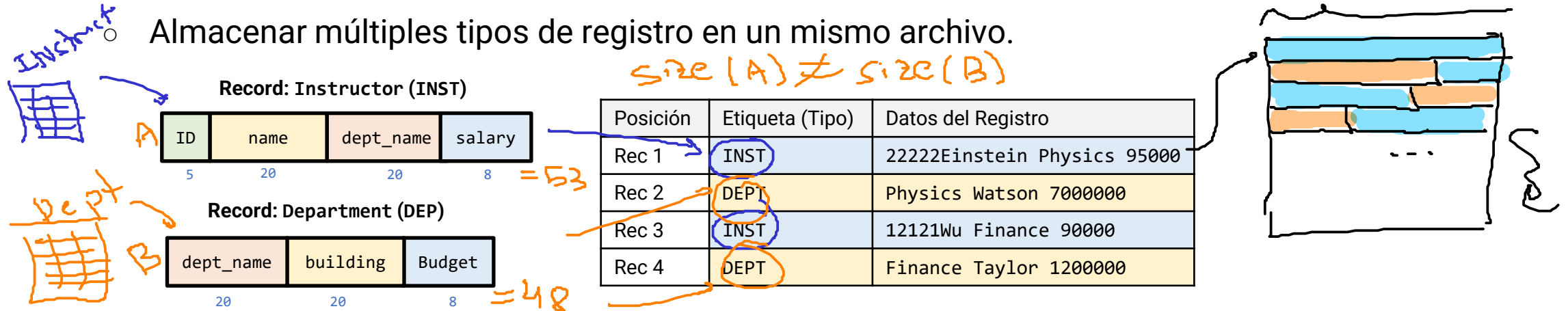
= 53 bytes



Registros de longitud variable (VLR)

- Los registros de longitud variable surgen en los sistemas de bases de datos de varias maneras:

- Almacenar múltiples tipos de registro en un mismo archivo.



- Tipos de registro que permiten longitudes variables en uno o más campos, por ejemplo cadenas (varchar).

```

type instructor = record
  ID : varchar(5);
  name : varchar(20);
  dept_name : varchar(20);
  salary : numeric(8,2);
end;

```

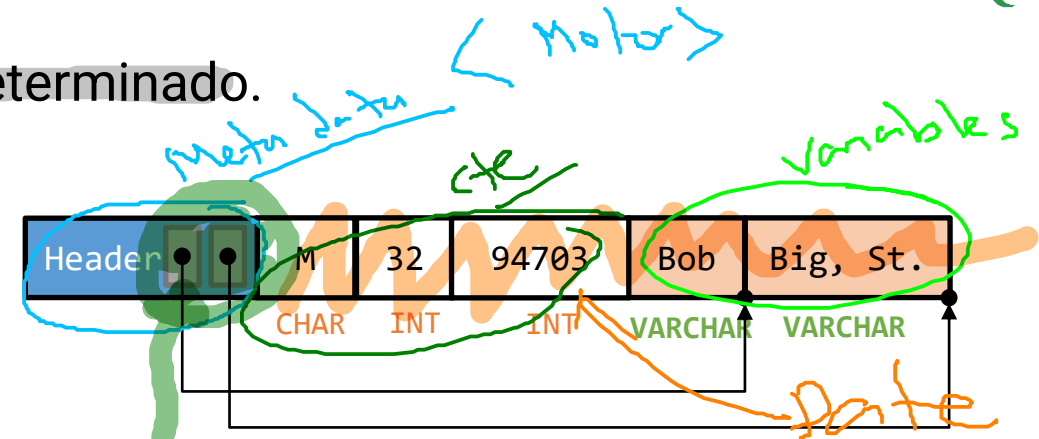
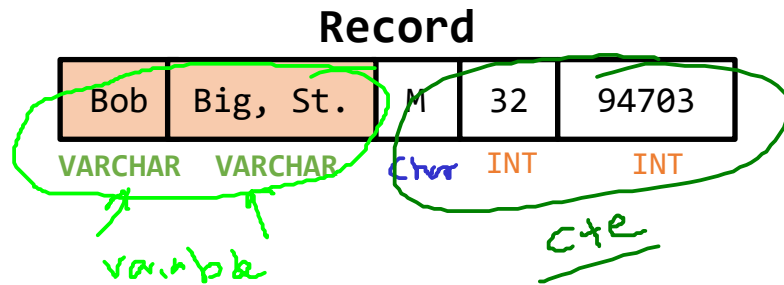
Handwritten notes: "extremo" and "caso" with arrows pointing to the record structure.

12121	Wu	Finance	90000
22222	Einstein	Physics	95000

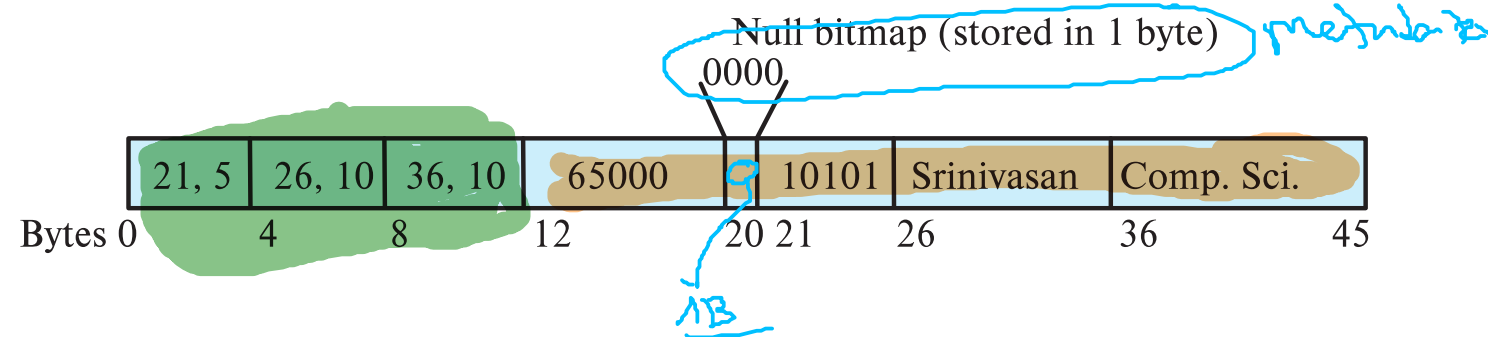
- Tipos de registro que permiten **campos repetidos** (utilizados en algunos modelos de datos más antiguos).

Registros de longitud variable (VLR)

- Los atributos se almacenan en un orden determinado.



- Los atributos de longitud variable se representan mediante una estructura de tamaño fijo (*offset, longitud*), mientras que los datos reales se almacenan después de los atributos de longitud fija.
- Los valores nulos se representan mediante un mapa de bits de valores nulos (null bitmap).



Registros de longitud variable (VLR)

Ejemplo: Retomemos la tabla de profesores, pero ahora con el esquema definido con varchar

- Si el registro del instructor "Einstein" comienza en la dirección de memoria 1000, ¿en qué dirección exacta comienza el campo salary? (**Rta:** 1012)
- Si quiero leer únicamente el dept_name, ¿cuántos bytes debe ignorar (saltar) el sistema desde el inicio del registro? (**Rta:** No)
- Si el nombre del instructor es "Wu" (solo 2 letras), ¿cuántos bytes ocupa ese registro en el disco? ¿Sigue midiendo 53 bytes o se reduce? (**Rta:** 35)
- Cuántos bytes de "relleno" (basura o espacios) se están desperdiciando en el campo name si guardamos a "Einstein" (8 letras)? (**Rta:** 0)
- Qué sucede si intentamos contratar a un instructor cuyo nombre tiene 25 letras? ¿El registro crece a 58 bytes o el nombre se corta? (**Rta:** Se corta)
- Si el campo salary es opcional (puede ser NULL), ¿el tamaño total del registro pasaría a ser de 45 bytes (53 - 8)? (**Rta:** No se reserva espacio).

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

```
type instructor = record
  ID : varchar(5);
  name : varchar(20);
  dept_name : varchar(20);
  salary : numeric(8,2);
end;
```

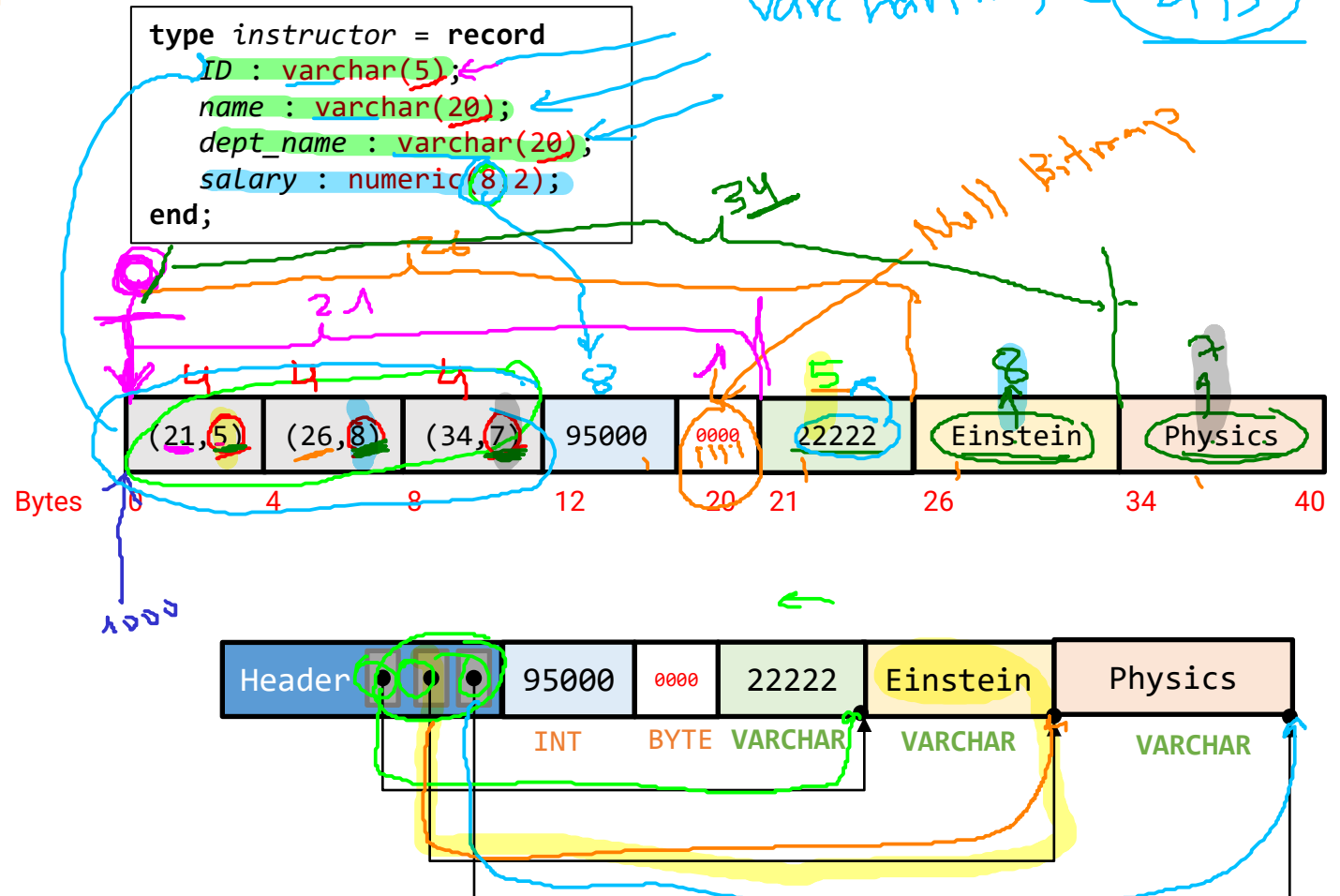
Registros de longitud variable (VLR)

Ejemplo: Retomemos la tabla de profesores, pero ahora con el esquema definido con varchar

32 bits = 4B
64 bits = 8B

4 campos (columnas)

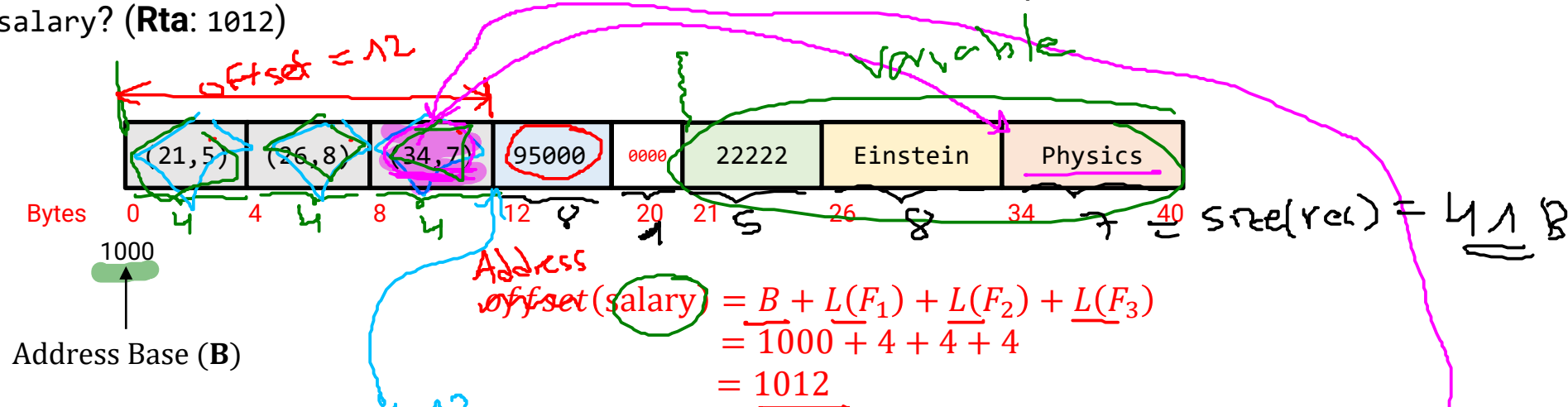
ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000



Registros de longitud variable (VLR)

Ejemplo: Retomemos la tabla de profesores, pero ahora con el esquema definido con varchar

- Si el registro del instructor "Einstein" comienza en la dirección de memoria 1000, ¿en qué dirección exacta comienza el campo salary? (**Rta:** 1012)



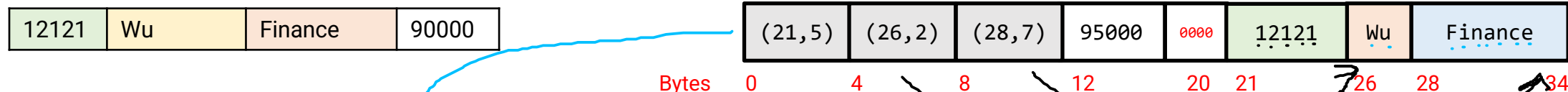
- Si quiero leer únicamente el dept_name, ¿cuántos bytes debe ignorar (saltar) el sistema desde el inicio del registro?
Rta: No es necesario, el sistema ahora realiza un paso extra (Indirección). Primero lee el puntero en los bytes 8-11 para obtener el (offset, longitud) y luego salta a la posición específica donde comienza el nombre del departamento en la sección de datos.

Registros de longitud variable (VLR)

Ejemplo: Retomemos la tabla de profesores, pero ahora con el esquema definido con varchar

- Si el nombre del instructor es "Wu" (solo 2 letras), ¿cuántos bytes ocupa ese registro en el disco? ¿Sigue midiendo 53 bytes o se reduce?

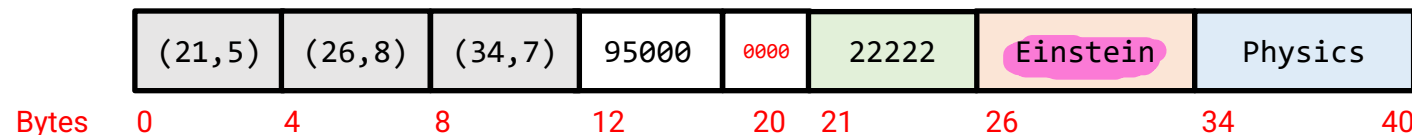
Rta: 35 bytes



$$\begin{aligned}
 L(\text{record}) &= L(F_1) + L(F_2) + L(F_3) + L(\text{salary}) + L(\text{BitMap}) + L(\text{ID}) + L(\text{name}) + L(\text{dept_name}) \\
 &= 4 + 4 + 4 + 8 + 1 + 5 + 2 + 7 \\
 &= 35
 \end{aligned}$$

- Cuántos bytes de "relleno" (basura o espacios) se están desperdiciando en el campo name si guardamos a "Einstein" (8 letras)?

Rta: No existe el relleno (varchar). El registro termina exactamente donde termina el último carácter del último campo variable.

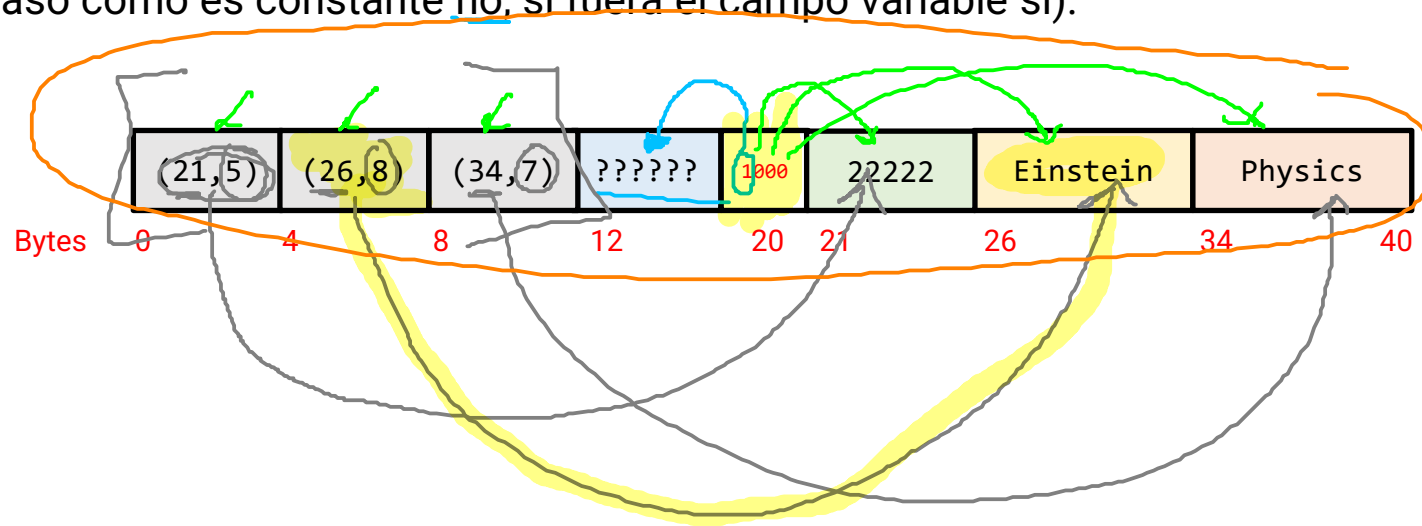


Registros de longitud variable (VLR)

Ejemplo: Retomemos la tabla de profesores, pero ahora con el esquema definido con varchar

- Si el campo salary es opcional (puede ser NULL), ¿el tamaño total del registro pasaría a ser de 45 bytes (53 - 8)?
(Rta: Depende del tipo de dato, en este caso como es constante no, si fuera el campo variable si).

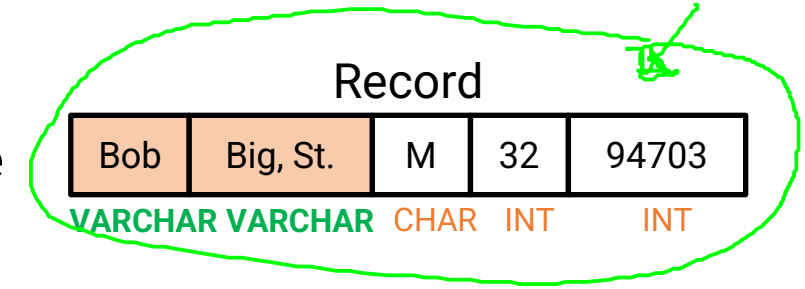
22222	Einstein	Physics	
-------	----------	---------	--



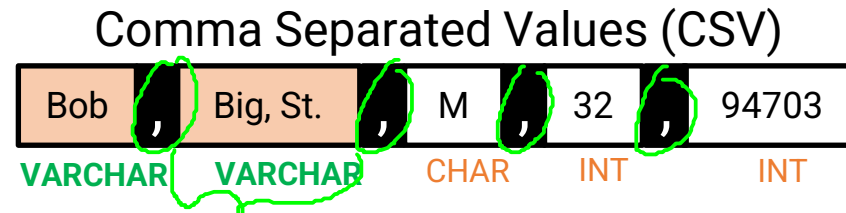
Registros de longitud variable (VLR)

Formatos

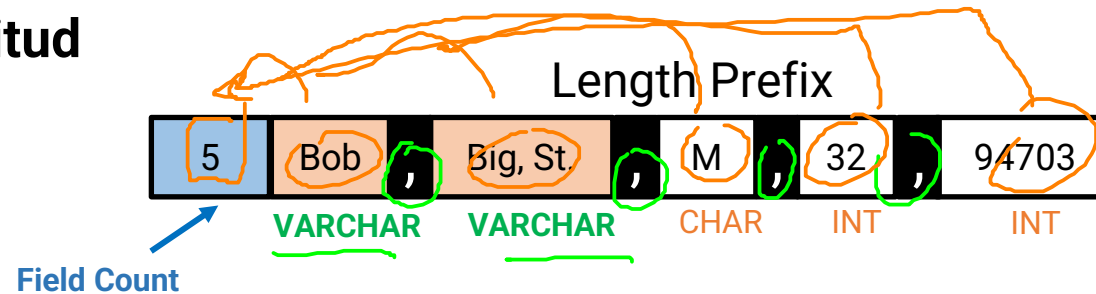
Los **formatos** determinan cómo se disponen los campos dentro de un único registro para permitir que el DBMS localice los datos. Existen varios formatos alternativos (Numero de campos fijo):



- Almacenamiento consecutivo con delimitadores

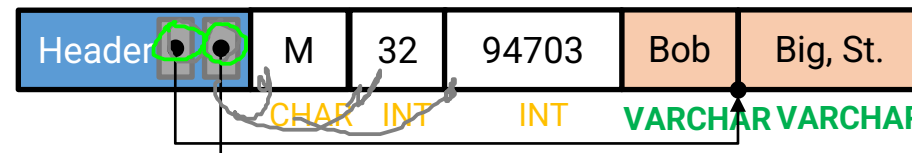


- Prefijo de longitud



- Arreglo de desplazamientos

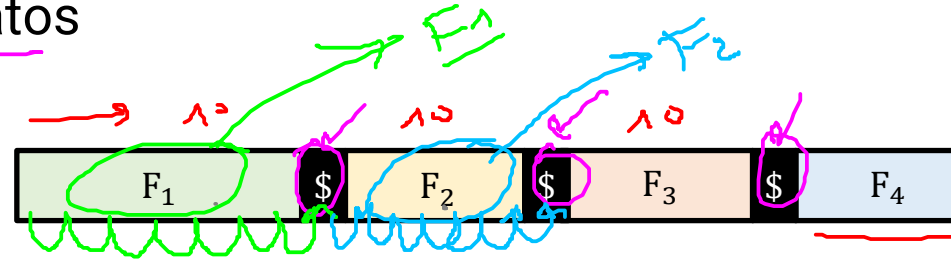
stated



Registros de longitud variable (VLR)

Almacenamiento consecutivo con delimitadores

- Los campos se guardan uno tras otro, separados por caracteres especiales (como \$) que no aparecen en los datos

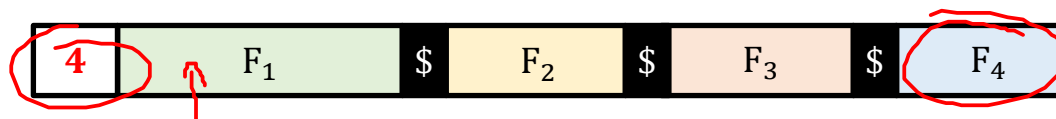


- Este formato requiere escanear todo el registro de forma secuencial para localizar un campo específico.

Registros de longitud variable (VLR)

Prefijo de longitud

- El registro comienza con bytes que indican su longitud total.

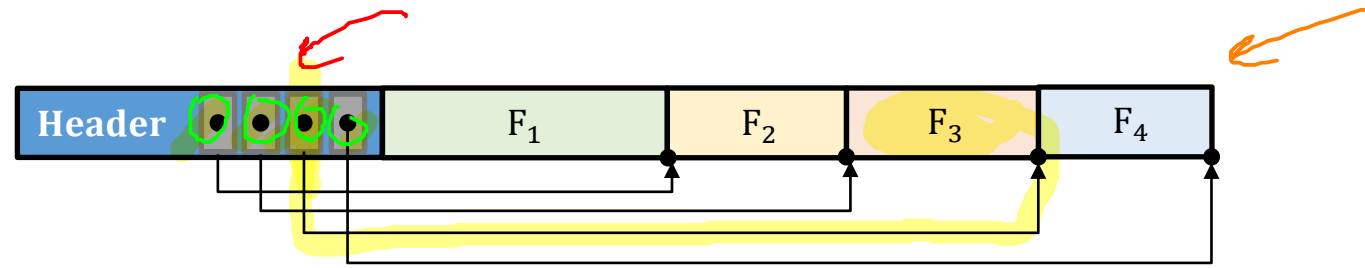


- La longitud ayuda al sistema a saber dónde termina un registro y comienza el siguiente

Registros de longitud variable (VLR)

Arreglo de desplazamientos (Offsets)

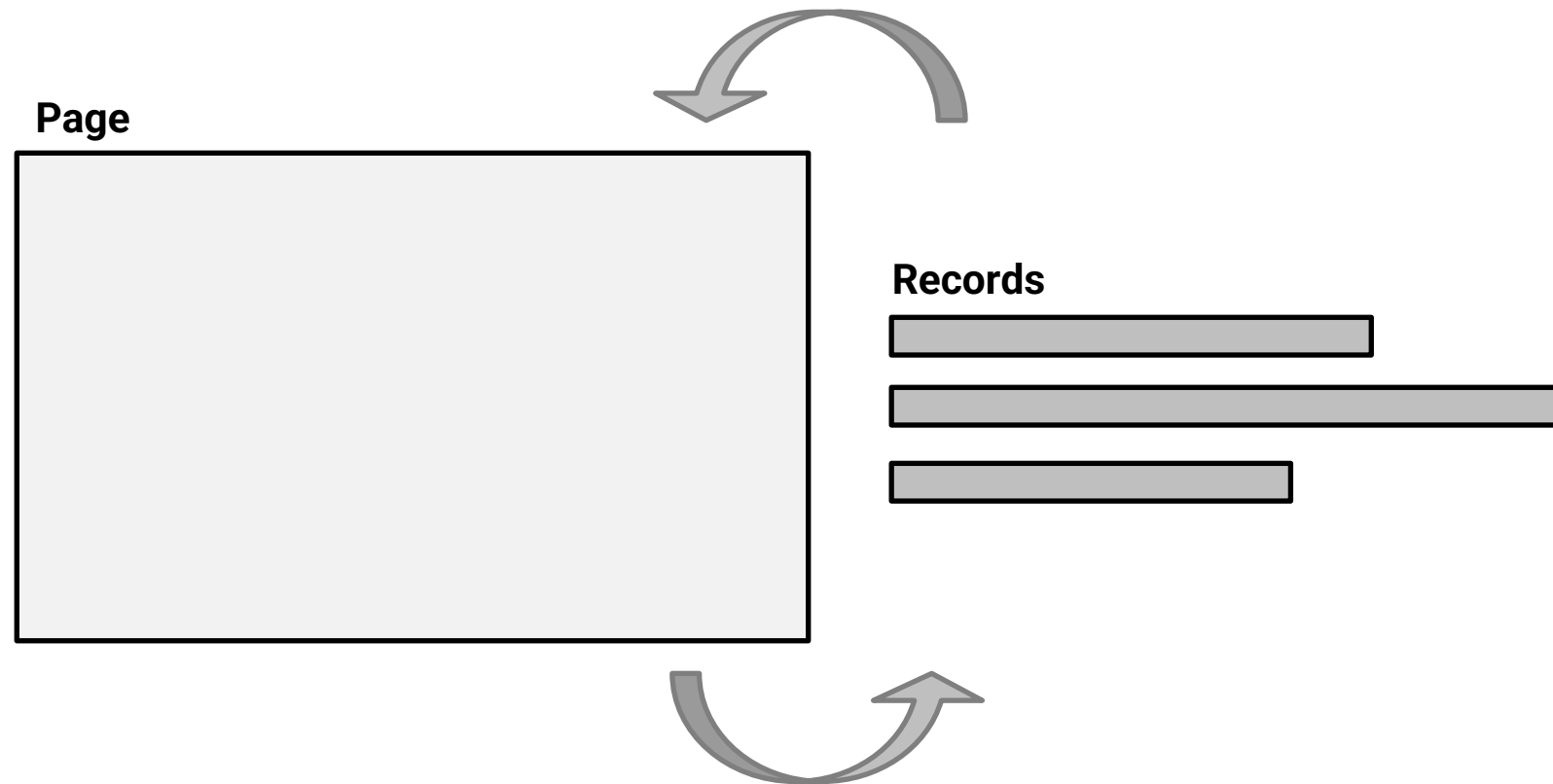
- Se reserva un espacio al inicio del registro para un arreglo de enteros que apuntan a la dirección de inicio de cada campo.



- Este método es superior porque permite el acceso directo a cualquier campo y ofrece una forma limpia de manejar valores nulos.
- También puede ser empleado para manejar registros de longitud fija (FLR)

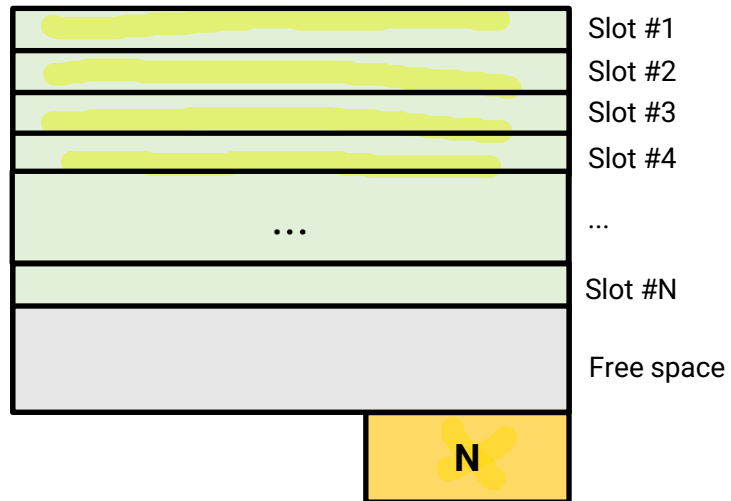
Formatos de pagina

- **Pregunta:** ¿Cómo almacenar registros en una página/archivo? de tal forma que:
 - Se pueda apuntar a cada uno de los registros
 - Sea posible insertar/eliminar registros empleando pocos accesos al disco

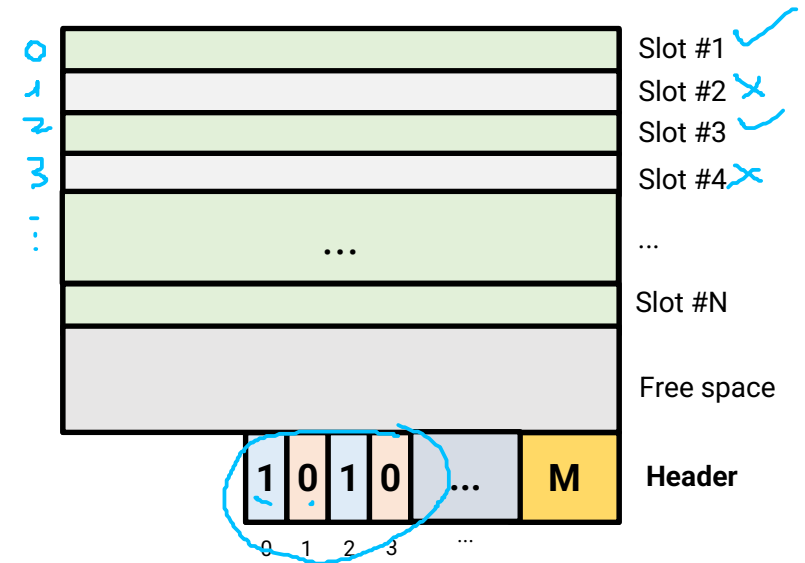


Formatos de pagina - FLR

Registros de Longitud Fija (FLR - Fixed Length Records)

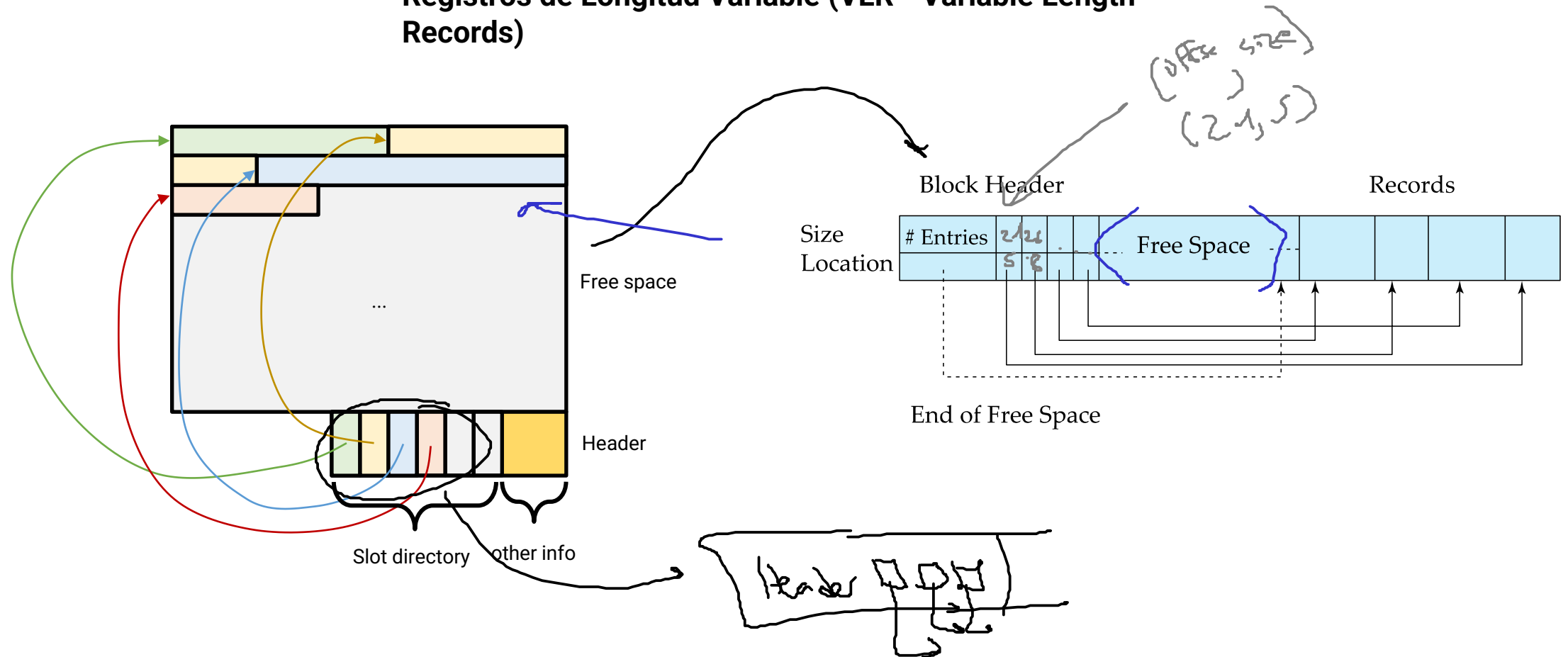


Registros de Longitud Fija (FLR - Fixed Length Records) – Con mapa de bits



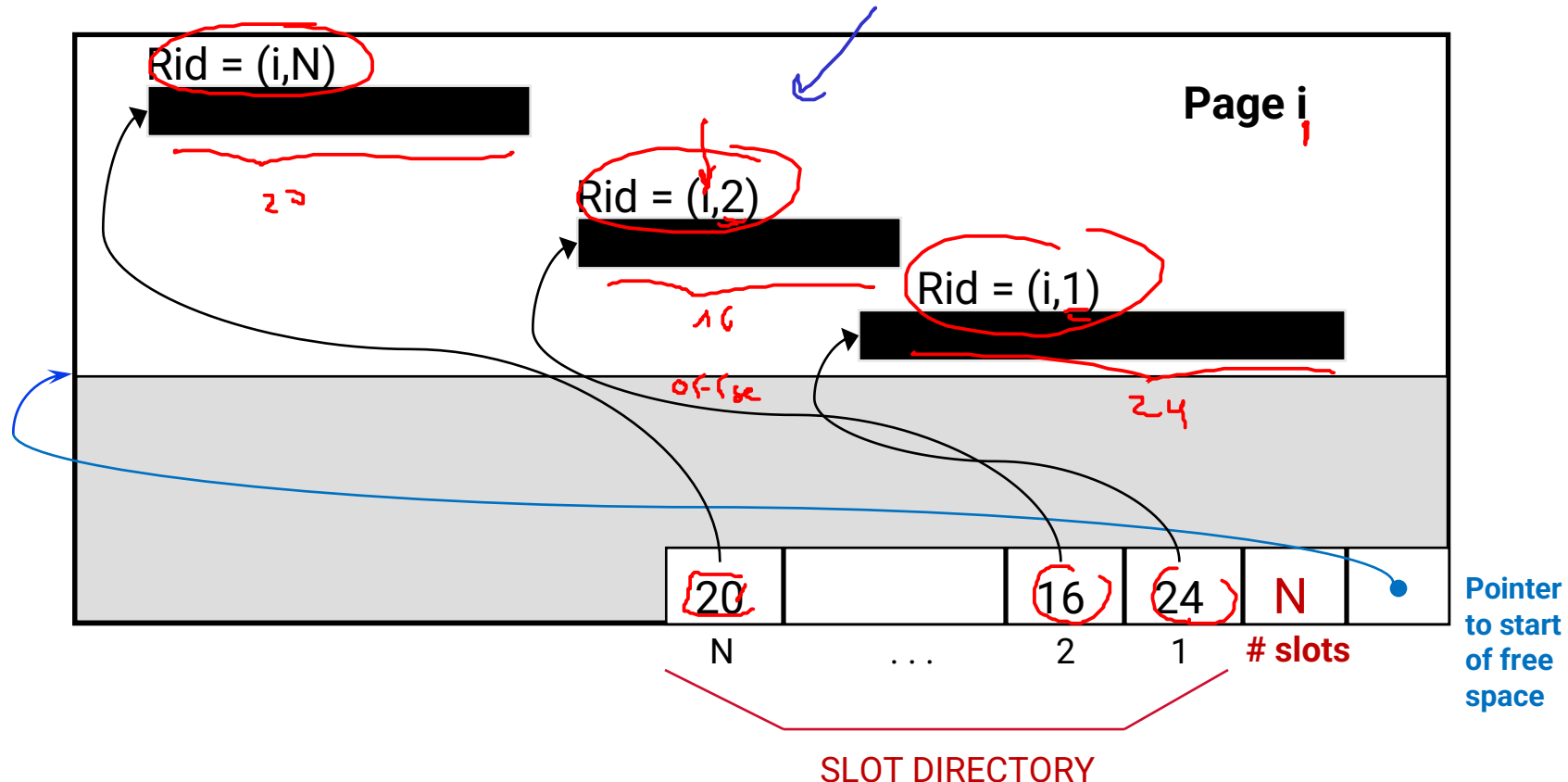
Formatos de pagina - VLR

Registros de Longitud Variable (VLR - Variable Length Records)



Slotted Page (Pagina ranurada)

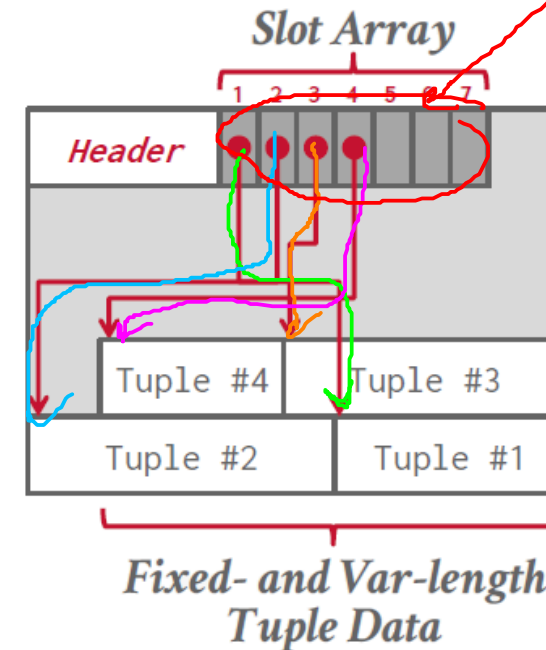
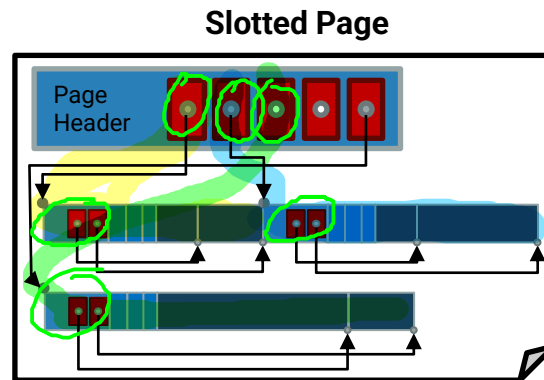
- Es el método más común y flexible para organizar registros de longitud variable dentro de una página de base de datos.
- Gestionar el espacio libre de forma eficiente y permitir que los registros se muevan físicamente dentro de la página sin perder su rastro



Slotted Page (Pagina ranurada)

Anatomía de la pagina

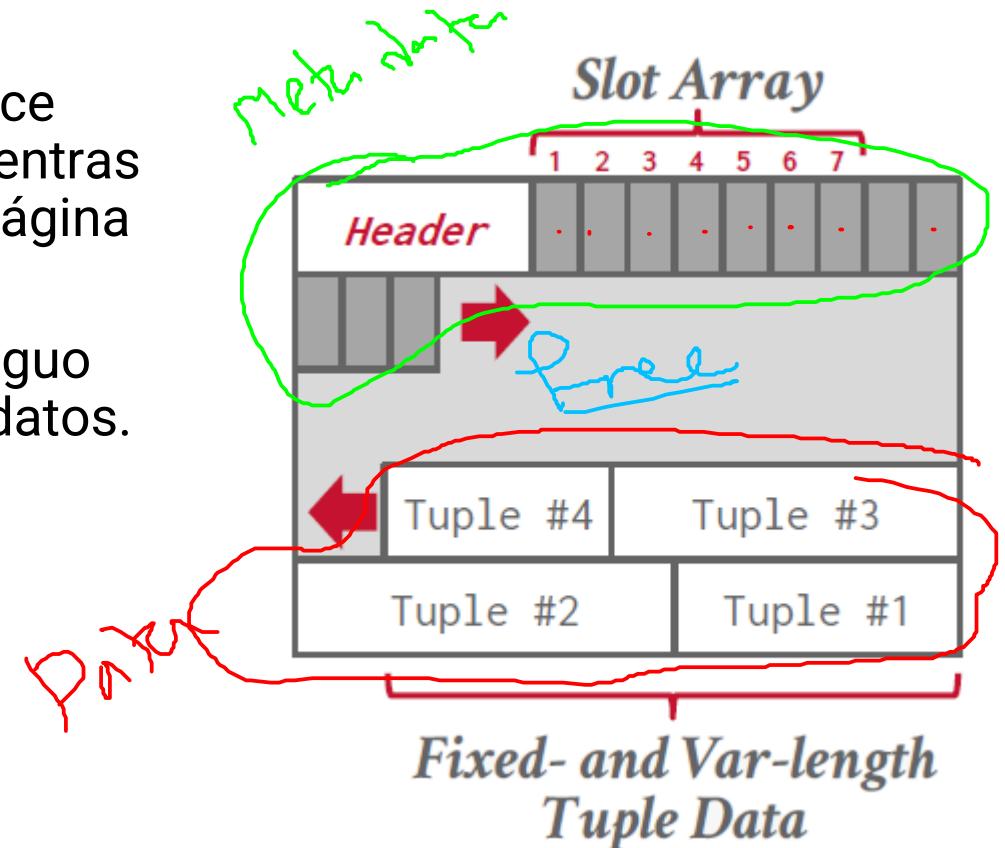
Los punteros no deben apuntar directamente al registro; en su lugar, deben apuntar a la entrada correspondiente del registro en la cabecera (el directorio de slots).



Slotted Page (Página ranurada)

Funcionamiento

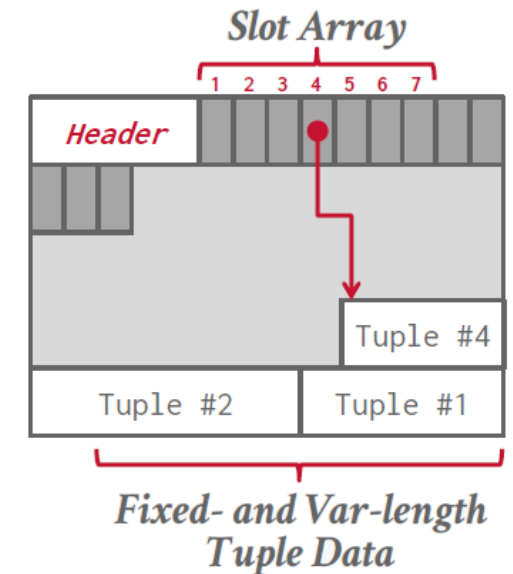
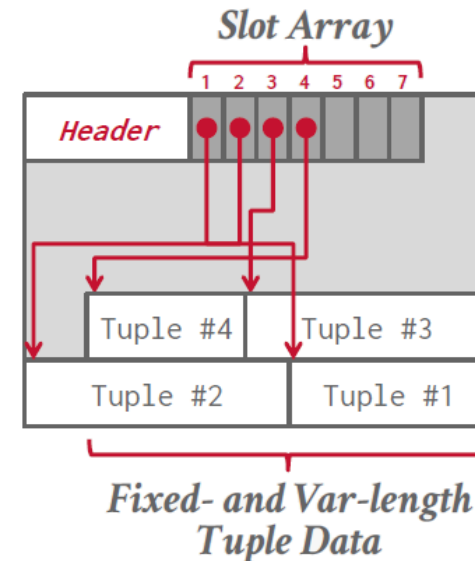
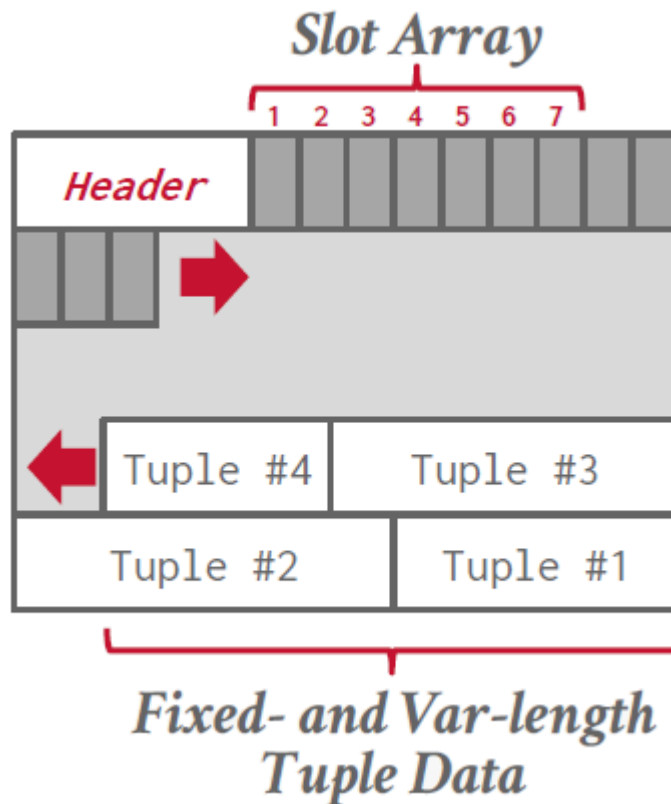
- **Crecimiento Opuesto:** El directorio de ranuras crece desde el principio de la página hacia adelante, mientras que los registros se insertan desde el final de la página hacia atrás.
- **Espacio Libre:** Se mantiene como un bloque contiguo en el centro de la página, entre el directorio y los datos.



Slotted Page (Pagina ranurada)

Funcionamiento

- Gracias a la estrategia de **crecimiento opuesto**, el espacio libre siempre queda concentrado en el medio, lo que permite aprovechar cada byte disponible antes de que la página se considere "llena".

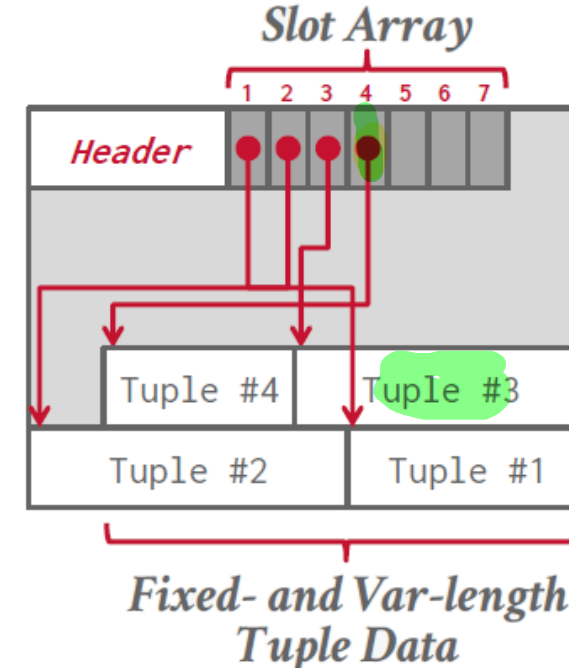
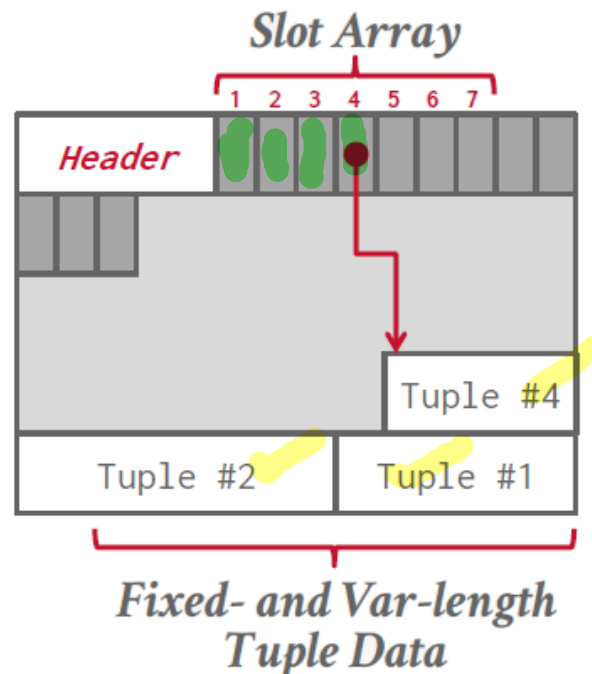


Slotted Page (Pagina ranurada)

Inserción de registros

Cuando se inserta un registro, el DBMS:

- Se coloca el **registro en el espacio libre de la página**
- agrega un **nuevo slot en el slot array**
- el slot guarda el **offset donde comienza el registro**

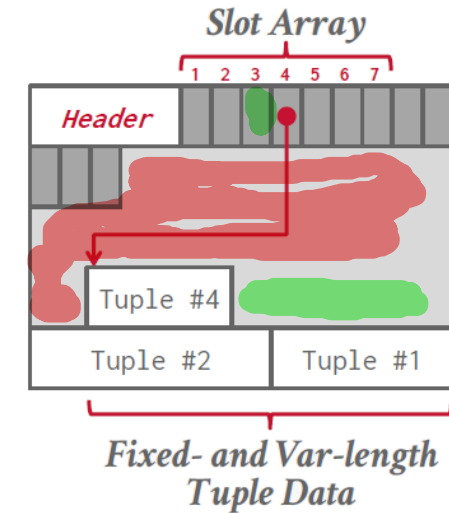
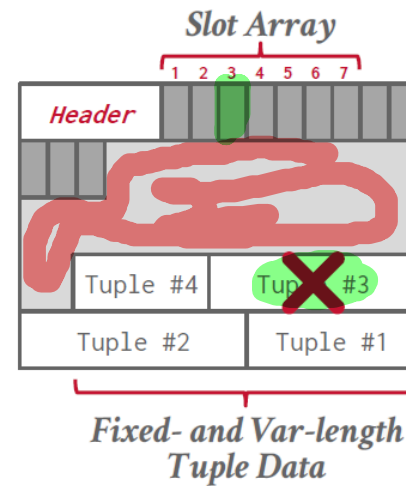
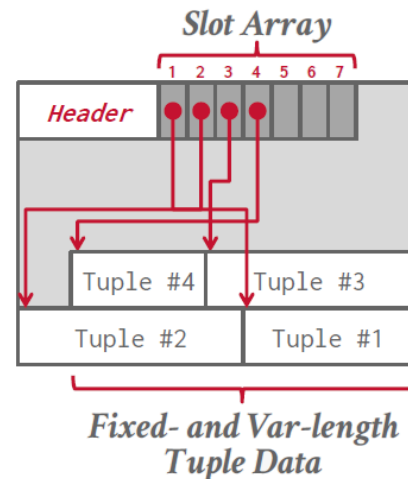


Slotted Page (Pagina ranurada)

Eliminación de registros

Cuando un registro se elimina:

- El slot se **marca como vacío**
- El espacio del registro queda **disponible**
- El resto de los registros **no necesitan moverse inmediatamente**

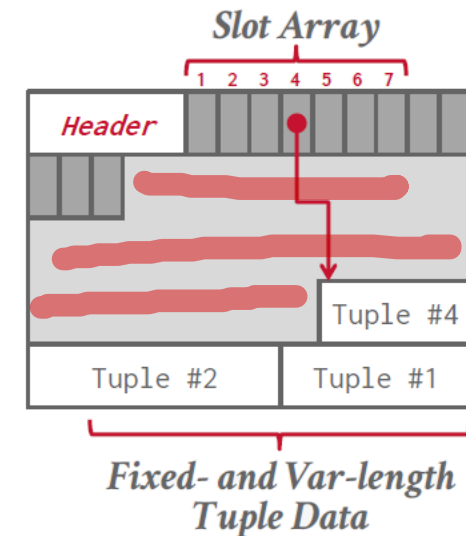
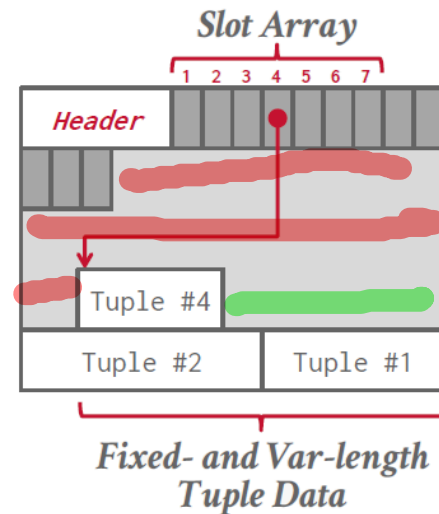
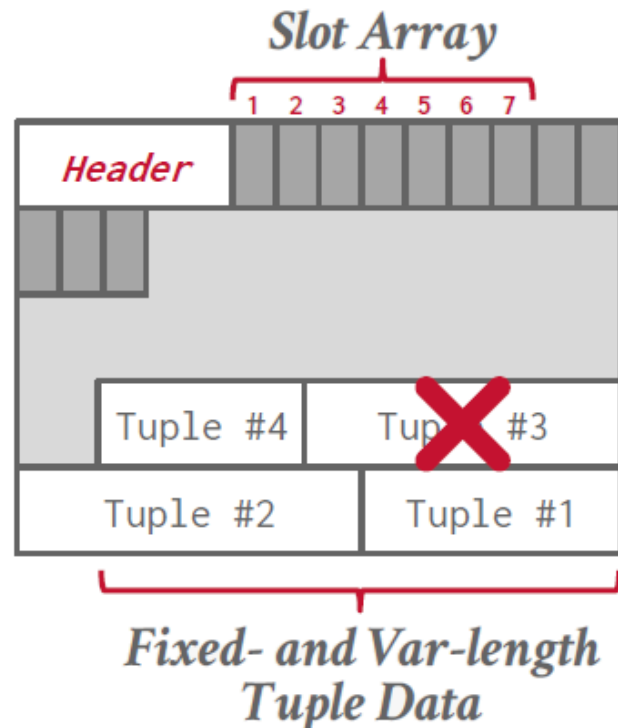


Slotted Page (Pagina ranurada)

Reorganización de registros

Cuando el espacio libre se fragmenta:

- El sistema puede **mover registros dentro de la página**
- Solo se actualizan los **offsets en el slot array**

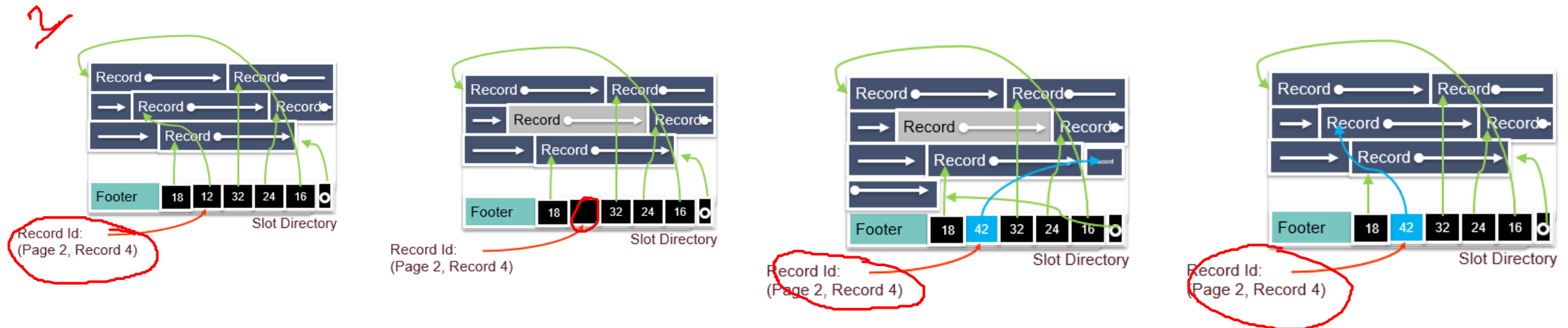


Slotted Page (Pagina ranurada)

Sobre el identificador de registro **RID**

El identificador de registro (RID) externo suele ser una pareja (ID de página, número de ranura).

- **Ventaja de Indirección:** Como el **RID** apunta a una ranura en el directorio y no a una posición física exacta, el DBMS puede mover registros dentro de la página para desfragmentar el espacio.
- **Resultado:** Solo hace falta actualizar el desplazamiento (offset) en el directorio; las referencias externas (índices) permanecen válidas.



Referencias

- <https://www.mongodb.com/resources/basics/databases/nosql-explained>
- <https://blog.bytebytego.com/p/sql-vs-nosql-choosing-the-right-database>
- <https://cs186berkeley.net/notes/note17/>
- <https://levelup.gitconnected.com/postgresql-deep-dive-the-anatomy-of-disk-file-and-page-layout-part-2-9b6fb7ede383>
- <https://medium.com/@pablo.lopez.santori/what-do-postgres-tables-actually-look-like-426028eabc1a>
- <https://michalpittr.substack.com/p/how-does-sqlite-store-data>
- <https://cs186berkeley.net/notes/note3/>
- <https://github.com/MIT-DB-Class/simple-db-hw>
- <https://simplifiedb-java.netlify.app/#/>
- <https://github.com/quazi-irfan/pySimpleDB>
- <https://tinydb.readthedocs.io/en/latest/>

UNIVERSIDAD DE ANTIOQUIA

Curso de Estructuras de datos
Clase 4 – Archivos (Parte 2)